

Department of Freshman Engineering

DATA STRUCTURES

Course Code: 2501CS03

Important Questions — Detailed Answers

Units 3 · 4 · 5

Topics Covered: Linked Lists | Stacks | Queues | Trees | Graphs

UNIT 3: LINKED LISTS

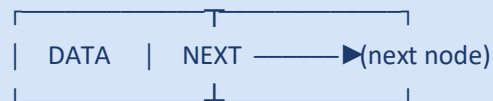
Q1. Operations in Singly Linked List (SLL) [8M]

What is a Singly Linked List?

A Singly Linked List (SLL) is a linear data structure where each element (called a node) contains two parts:

- DATA: Stores the actual value
- NEXT: A pointer/reference to the next node in the sequence
- The last node's NEXT pointer points to NULL, indicating end of list
- A HEAD pointer always points to the first node of the list

Node Structure



Each node holds data + pointer to next node

a) Insert Element at the Beginning

Algorithm: Create a new node → Set new node's NEXT = HEAD → Update HEAD = new node

Insert at Beginning: New node becomes new HEAD

BEFORE:

HEAD



AFTER (inserted 5):

HEAD



Fig 1.1: Insert at Beginning — new node becomes the new HEAD

Algorithm: Insert at Beginning

Step 1: Create a new node with given data

Step 2: Set newNode->next = HEAD

Step 3: Set HEAD = newNode

Step 4: If list is empty, HEAD = newNode and newNode->next = NULL

Time Complexity: $O(1)$ | Space Complexity: $O(1)$

b) Insert Element at the End

Algorithm: Create a new node → Traverse to last node → Set last node's NEXT = new node

Insert at End: Traverse to last node, update its next pointer

BEFORE:



AFTER (inserted 40 at end):

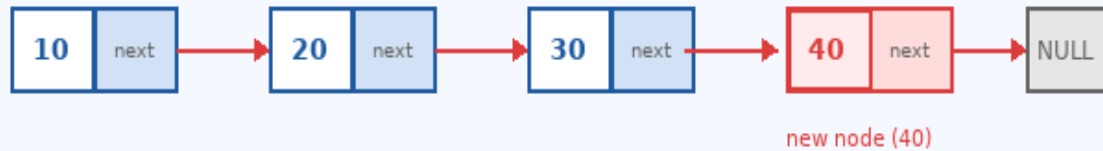


Fig 1.2: Insert at End — traverse to last node, link new node

Algorithm: Insert at End

Step 1: Create a new node with given data, set newNode->next = NULL

Step 2: If list is empty (HEAD == NULL), set HEAD = newNode and return

Step 3: Set temp = HEAD

Step 4: Traverse: while(temp->next != NULL) temp = temp->next

Step 5: Set temp->next = newNode

Time Complexity: $O(n)$ | Space Complexity: $O(1)$

c) Delete the Specified Element

Algorithm: Search for the element → Re-link the previous node's NEXT to skip the deleted node → Free memory

Delete Specified Element: Re-link previous node's next pointer

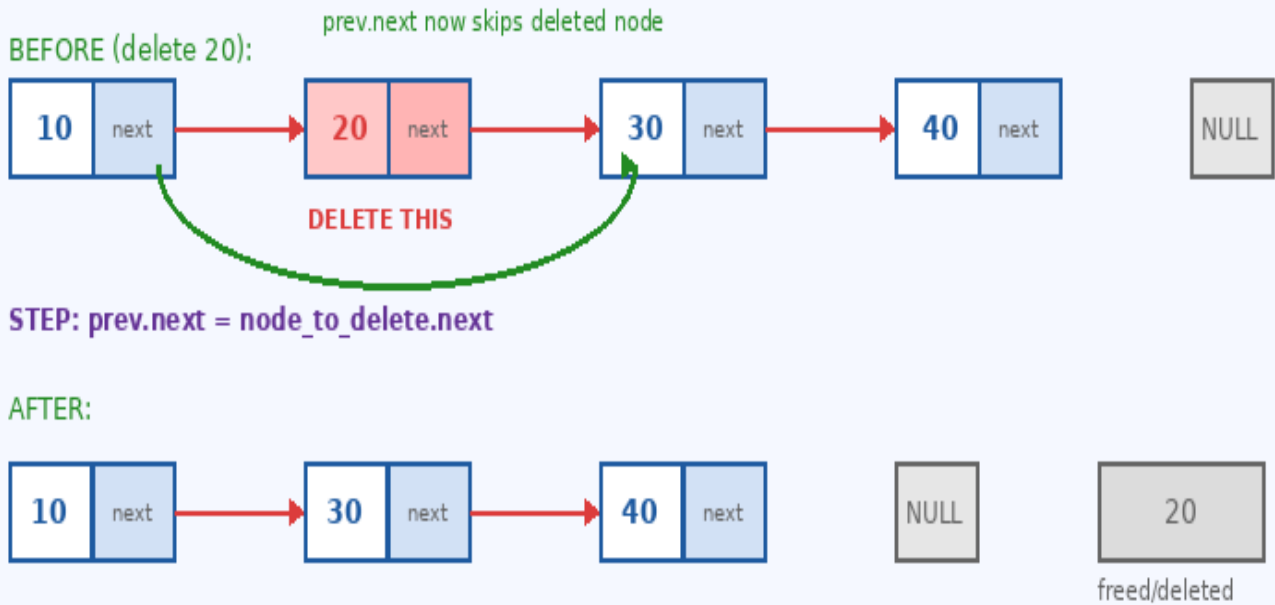


Fig 1.3: Delete specified element — bypass node and free memory

Algorithm: Delete Specified Element

Step 1: If $\text{HEAD} \rightarrow \text{data} == \text{key}$: $\text{HEAD} = \text{HEAD} \rightarrow \text{next}$; $\text{free}(\text{temp})$; return

Step 2: Set $\text{prev} = \text{HEAD}$, $\text{curr} = \text{HEAD} \rightarrow \text{next}$

Step 3: Traverse: while($\text{curr} \neq \text{NULL} \ \&\& \ \text{curr} \rightarrow \text{data} \neq \text{key}$)
 $\text{prev} = \text{curr}$; $\text{curr} = \text{curr} \rightarrow \text{next}$

Step 4: If $\text{curr} == \text{NULL}$: Element not found

Step 5: $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$ (bypass the node)

Step 6: $\text{free}(\text{curr})$

Time Complexity: $O(n)$ | Space Complexity: $O(1)$

d) Search the Given Element

Algorithm: Start from HEAD $\rightarrow$ Compare each node's data with the key $\rightarrow$ Return position if found, else NULL

Search Element: Traverse from HEAD, compare each node's data

Searching for 30:

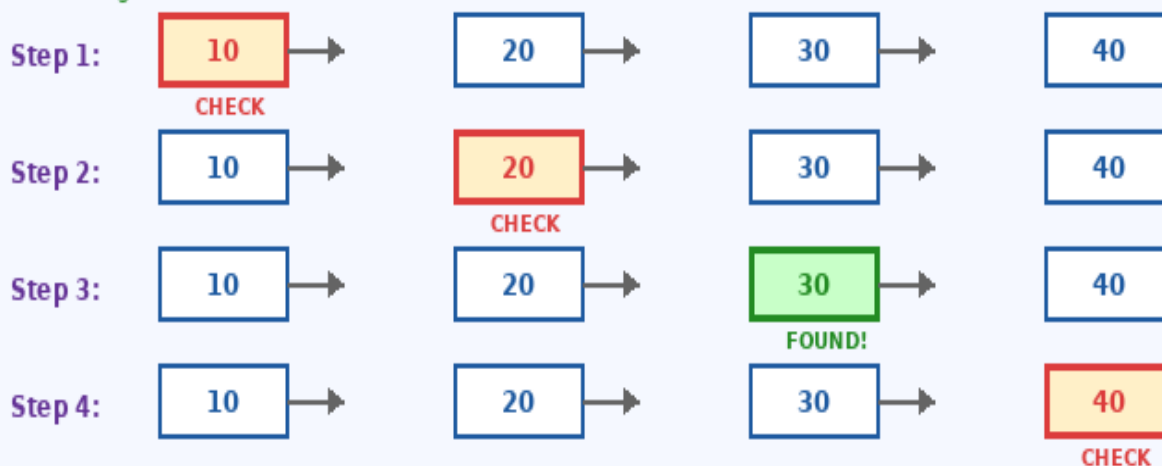


Fig 1.4: Sequential search — compare each node until found

Algorithm: Search Element

Step 1: Set temp = HEAD, position = 1

Step 2: while(temp != NULL):

 if(temp->data == key) return position ← FOUND

 temp = temp->next

 position++

Step 3: return -1 ← NOT FOUND

Time Complexity: $O(n)$ | Space Complexity: $O(1)$

Q2. Operations on Doubly Linked List (DLL) [8M]

What is a Doubly Linked List?

A Doubly Linked List is an advanced linked list where each node has THREE fields:

- PREV: Pointer to the previous node
- DATA: The actual value stored
- NEXT: Pointer to the next node

Advantage over SLL: Can traverse in BOTH directions (forward and backward).

Doubly Linked List Node Structure

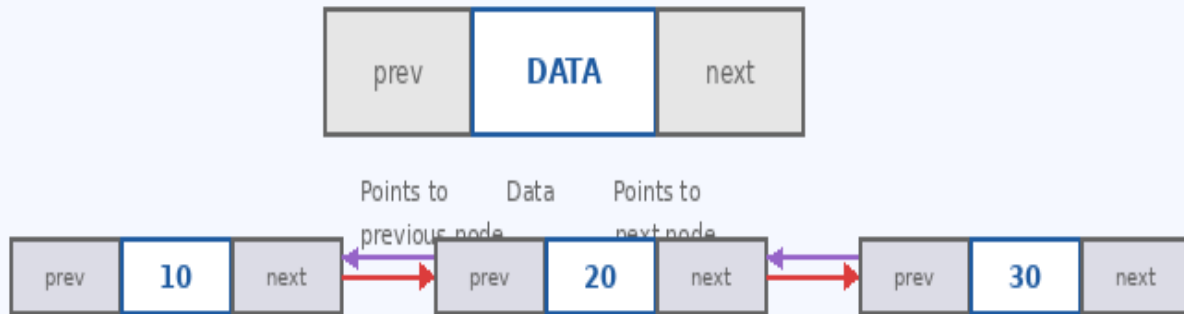


Fig 2.1: DLL node structure — bidirectional pointers

i) Add at Beginning

`new->next = HEAD; HEAD->prev = new; HEAD = new`



HEAD→

ii) Delete at Beginning

`temp = HEAD; HEAD = HEAD->next; HEAD->prev = NULL; free(temp)`



After: HEAD→20, 20->prev=NULL

iii) Add after an item

`new->next=p->next; new->prev=p; p->next->prev=new; p->next=new`



Insert 15 between 10 and 20

iv) Delete from End

`Traverse to last; last->prev->next = NULL; free(last)`



Fig 2.2: All four DLL operations visualized

i) Add an element at the Beginning

```

newNode->data = value
newNode->next = HEAD
newNode->prev = NULL
if(HEAD != NULL) HEAD->prev = newNode
HEAD = newNode

```

Time Complexity: $O(1)$ — No traversal needed, constant time operation.

ii) Delete an element at the Beginning

```

if(HEAD == NULL) return // Empty list
temp = HEAD
HEAD = HEAD->next
if(HEAD != NULL) HEAD->prev = NULL
free(temp)

```

Time Complexity: $O(1)$ — Direct access to HEAD node.

iii) Add an element after an item

```

// Insert newNode after node 'p'
newNode->next = p->next
newNode->prev = p
if(p->next != NULL) p->next->prev = newNode
p->next = newNode

```

Time Complexity: $O(n)$ for finding node p ; $O(1)$ for insertion.

iv) Delete an element from the End

```

if(HEAD == NULL) return // Empty list
if(HEAD->next == NULL) { free(HEAD); HEAD=NULL; return }
temp = HEAD
while(temp->next != NULL) temp = temp->next // Go to last node
temp->prev->next = NULL
free(temp)

```

Time Complexity: $O(n)$ — Must traverse to last node.

Q3. a) Reversing a Singly Linked List | b) Applications of Linked Lists [4M + 4M]

a) Reversing a Singly Linked List

To reverse a singly linked list, we use THREE pointer variables: prev, curr, and next. The idea is to change each node's NEXT pointer to point to its PREVIOUS node.

Reversing a Singly Linked List

Use three pointers: prev=NULL, curr=HEAD, next=curr->next

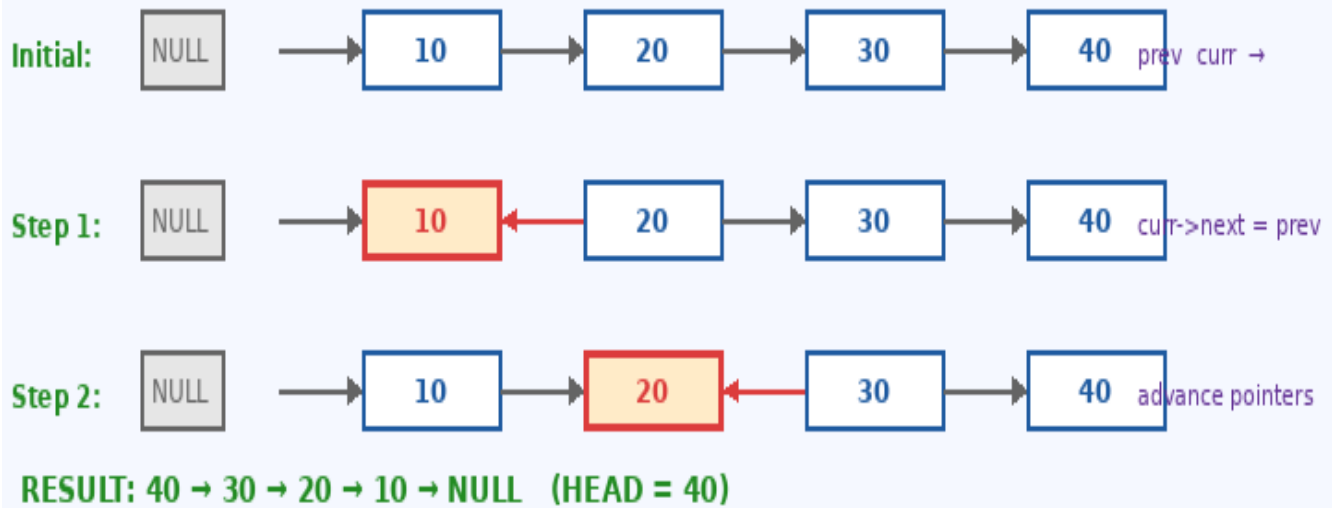


Fig 3.1: Step-by-step reversal using three pointers

Algorithm:

```

REVERSE (HEAD) :
prev = NULL
curr = HEAD
while(curr != NULL):
    next = curr->next    // Save next node
    curr->next = prev    // Reverse the pointer
    prev = curr          // Move prev forward
    curr = next          // Move curr forward
HEAD = prev             // prev is now the new HEAD
  
```

Time Complexity: $O(n)$ — Single pass through the list

Space Complexity: $O(1)$ — Only 3 extra pointers used

b) Applications of Linked List

Application	Description
Dynamic Memory Allocation	Memory allocation using free-block lists; no pre-allocation needed
Implementation of Stacks/Queues	Stacks and queues are implemented using linked lists for dynamic sizing
Polynomial Representation	Each term (coefficient, exponent) stored as a node for arithmetic operations
Browser History	Each visited URL is a node; back/forward navigation uses DLL

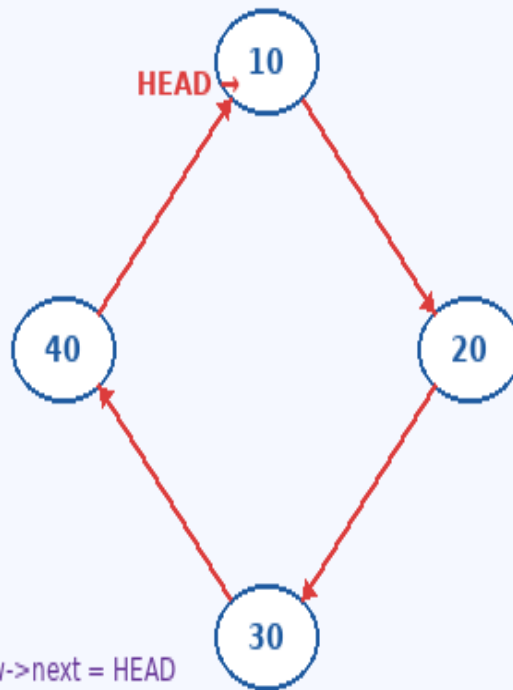
Music Playlist	Songs linked in sequence; circular DLL enables repeat and shuffle
Undo/Redo Operations	Each action stored as node; traversal gives undo/redone capability
Symbol Table in Compilers	Variable names and attributes stored in linked list for fast lookup
Adjacency List for Graphs	Graph edges represented as linked list from each vertex

Q4. Operations on Circular Linked List (CLL) [8M]

A Circular Linked List is a variation where the LAST node's NEXT pointer points back to the FIRST (HEAD) node instead of NULL. This forms a circular chain.

- No node has a NULL pointer (except in an empty list)
- Any node can be used as a starting point
- Useful for applications like round-robin scheduling

Circular Linked List - Last node points back to HEAD



Insert: Update LAST->next = new; new->next = HEAD

Delete: Find node, update prev->next = node->next **No NULL pointer - circular!**

Fig 4.1: Circular Linked List — last node points back to HEAD

Standard Operations on CLL

1. INSERT AT BEGINNING: Create node → node->next=HEAD → LAST->next=node → HEAD=node
2. INSERT AT END: Create node → traverse to LAST → LAST->next=node → node->next=HEAD

- | | |
|-----------------------|---|
| 3. INSERT AFTER NODE: | Create node → node->next=p->next → p->next=node |
| 4. DELETE FROM BEGIN: | temp=HEAD → HEAD=HEAD->next → LAST->next=HEAD → free(temp) |
| 5. DELETE FROM END: | Traverse to LAST and prev → prev->next=HEAD → free(LAST) |
| 6. DISPLAY: | Start at HEAD → print data → move to next → stop when back at HEAD |
| 7. SEARCH: | Traverse from HEAD → compare data → stop at HEAD again if not found |

Pseudo Code for Insert at End (CLL):

```
INSERT_END(HEAD, value):
    newNode = allocate node; newNode->data = value
    if(HEAD == NULL):           // Empty list
        HEAD = newNode
        newNode->next = HEAD
        return
    temp = HEAD
    while(temp->next != HEAD):  // Traverse to last node
        temp = temp->next
    temp->next = newNode        // Link last to new node
    newNode->next = HEAD        // New node points back to HEAD
```

Pseudo Code for Display (CLL):

```
DISPLAY(HEAD):
    if(HEAD == NULL): print 'List is empty'; return
    temp = HEAD
    do:
        print temp->data
        temp = temp->next
    while(temp != HEAD)        // Stop when we loop back to HEAD
```

UNIT 4: STACKS AND QUEUES

Q1. a) Stack using Array | b) Postfix Expression Evaluation [4M + 4M]

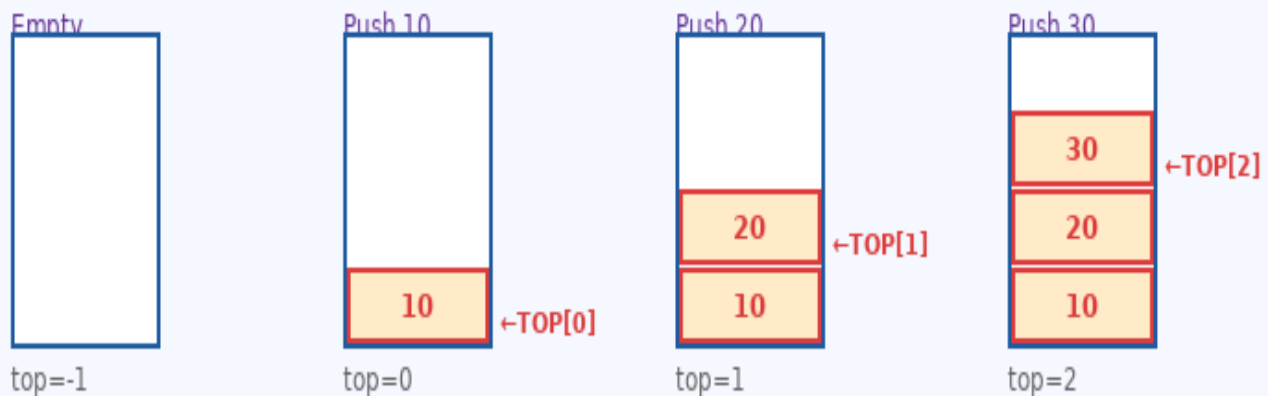
a) Stack using Array Representation

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The element inserted LAST is removed FIRST.

- PUSH: Insert element at the top
- POP: Remove element from the top
- PEEK/TOP: View top element without removing
- isEmpty: Check if stack is empty ($\text{top} == -1$)
- isFull: Check if stack is full ($\text{top} == \text{MAX}-1$)

Stack using Array - LIFO (Last In First Out)

PUSH Operations:



POP: Remove top element | Underflow if $\text{top} == -1$ | Overflow if $\text{top} == \text{MAX}-1$

PUSH: if $\text{top} == \text{MAX}-1 \rightarrow$ Overflow Error else $\rightarrow \text{stack}[\text{++top}] = \text{data}$
 POP: if $\text{top} == -1 \rightarrow$ Underflow Error else $\rightarrow \text{return stack}[\text{top--}]$

Fig 5.1: Stack array representation — PUSH operations and conditions

Algorithm — PUSH Operation:

```
PUSH(stack[], top, MAX, value):
    if(top == MAX - 1):
        print 'Stack OVERFLOW - Cannot push'
        return
```

```

top = top + 1
stack[top] = value
print 'Pushed:', value

```

Algorithm — POP Operation:

```

POP(stack[], top):
    if(top == -1):
        print 'Stack UNDERFLOW - Cannot pop'
        return -1
    value = stack[top]
    top = top - 1
    return value

```

Operation	Time Complexity
PUSH	O(1)
POP	O(1)
PEEK	O(1)
isEmpty/isFull	O(1)
Search	O(n)

b) Evaluation of Postfix Arithmetic Expression using Stack

A postfix expression (also called Reverse Polish Notation) has operators AFTER their operands. Stacks are used to evaluate postfix expressions efficiently.

Rules for Postfix Evaluation
1. Scan the expression from LEFT to RIGHT
2. If current character is an OPERAND → PUSH it onto the stack
3. If current character is an OPERATOR (+, -, *, /) → - POP top two elements: op2 = pop(), op1 = pop() - Compute result = op1 OPERATOR op2 - PUSH result back onto stack
4. At end of expression, stack top = FINAL RESULT

Postfix Evaluation: $ABC*+D+$ where $A=2, B=3, C=4, D=5$ Scan left to right: operand $\rightarrow$ push, operator $\rightarrow$ pop two, compute, push result

Symbol	Action	Stack	Compute
A(2)	Push 2	[2] $\leftarrow$ top	
B(3)	Push 3	[2, 3] $\leftarrow$ top	
C(4)	Push 4	[2, 3, 4] $\leftarrow$ top	
*	Pop 4, 3 $\rightarrow 3*4=12 \rightarrow$ Push 12	[2, 12] $\leftarrow$ top	$3 \times 4 = 12$
+	Pop 12, 2 $\rightarrow 2+12=14 \rightarrow$ Push 14	[14] $\leftarrow$ top	$2+12=14$
D(5)	Push 5	[14, 5] $\leftarrow$ top	
+	Pop 5, 14 $\rightarrow 14+5=19 \rightarrow$ Push 19 (RESULT)	[19] $\leftarrow$ top	$14+5=19$

FINAL RESULT = 19Fig 5.2: Postfix evaluation trace — $ABC*+D+$ where $A=2, B=3, C=4, D=5 \rightarrow \text{Result}=19$ **Q2. a) Queue with Enqueue/Dequeue | b) Circular Queue Operations [4M + 4M]****a) Queue — Enqueue and Dequeue Operations**

A Queue is a linear data structure following FIFO (First In, First Out). Elements are inserted at the REAR and removed from the FRONT.

Queue - FIFO (First In First Out) using Array

ENQUEUE (rear advances):



DEQUEUE (front advances):

Queue: [10, 20, 30] front=0, rear=2



↑ Dequeue 10 (front++) ↑ After: [20, 30] front=1

Enqueue: if rear == MAX-1 → Overflow else → arr[++rear] = data

Dequeue: if front > rear → Underflow else → return arr[front++]

Problem: Even if space exists at front, rear == MAX-1 shows overflow → Solved by Circular Queue

Fig 6.1: Queue — Enqueue adds at rear, Dequeue removes from front

Algorithm — Enqueue:

```
ENQUEUE(queue[], front, rear, MAX, value):
    if(rear == MAX - 1):
        print 'Queue OVERFLOW'
        return
    if(front == -1) front = 0 // First element
    rear = rear + 1
    queue[rear] = value
```

Algorithm — Dequeue:

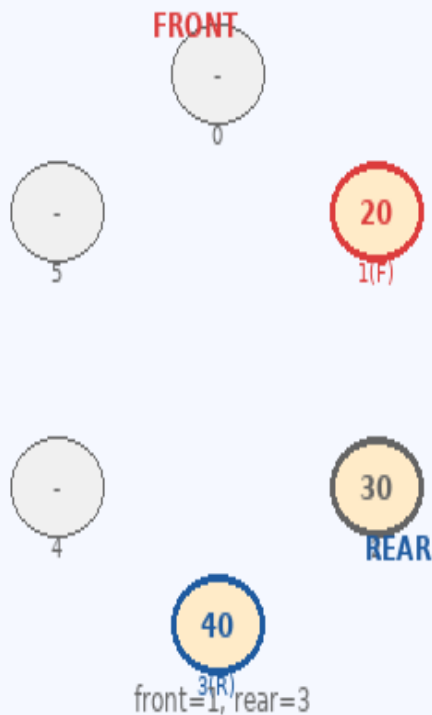
```
DEQUEUE(queue[], front, rear):
    if(front == -1 || front > rear):
        print 'Queue UNDERFLOW'
        return -1
    value = queue[front]
    front = front + 1
    if(front > rear) front = rear = -1 // Reset when empty
    return value
```

Problem with Linear Queue: Even after dequeue, the front space cannot be reused → Solved by Circular Queue.

b) Circular Queue — Enqueue and Dequeue

In a Circular Queue, the last position is connected back to the first, forming a circle. This solves the wasted space problem of linear queues.

Circular Queue - Wraps around to reuse freed space



Circular Queue Formulas:

$\text{rear} = (\text{rear} + 1) \% \text{MAX}$
 $\text{front} = (\text{front} + 1) \% \text{MAX}$

ENQUEUE:

if $(\text{front} == (\text{rear} + 1) \% \text{MAX}) \rightarrow \text{FULL}$
 else $\text{rear} = (\text{rear} + 1) \% \text{MAX}$
 $\text{queue}[\text{rear}] = \text{data}$

DEQUEUE:

if $(\text{front} == \text{rear} + 1 \text{ or empty}) \rightarrow \text{EMPTY}$
 else $\text{front} = (\text{front} + 1) \% \text{MAX}$
 return $\text{queue}[\text{front}]$

EMPTY: $\text{front} == (\text{rear} + 1) \% \text{MAX}$

FULL: $\text{front} == (\text{rear} + 2) \% \text{MAX}$

Fig 6.2: Circular Queue structure and formulas for wrap-around indexing

Algorithm — Circular Queue Enqueue:

```

CQ_ENQUEUE(cq[], front, rear, MAX, value):
    nextRear = (rear + 1) % MAX
    if(nextRear == front):
        print 'Circular Queue FULL'
        return
    if(front == -1) front = 0
    rear = nextRear
    cq[rear] = value
  
```

Algorithm — Circular Queue Dequeue:

```

CQ_DEQUEUE(cq[], front, rear, MAX):
    if(front == -1):
        print 'Circular Queue EMPTY'
        return -1
    value = cq[front]
    if(front == rear) front = rear = -1 // Queue becomes empty
    else front = (front + 1) % MAX
    return value
  
```

Q3. a) Circular Queue Operations | b) Types of Queues [4M + 4M]

a) Operations on Circular Queue with Diagrams

A Circular Queue uses modulo arithmetic to wrap around. Key conditions:

- EMPTY condition: $\text{front} == -1$ OR $\text{front} == (\text{rear} + 1) \% \text{MAX}$
- FULL condition: $(\text{rear} + 1) \% \text{MAX} == \text{front}$
- Enqueue: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$; $\text{cq}[\text{rear}] = \text{data}$
- Dequeue: $\text{data} = \text{cq}[\text{front}]$; $\text{front} = (\text{front} + 1) \% \text{MAX}$
- Advantage: Reuses empty slots created after dequeue, unlike linear queue
- Space: $O(n)$; Enqueue/Dequeue: $O(1)$

Property	Circular Queue vs Linear Queue
Space Utilization	Circular: Full reuse of array Linear: Wasted front space
FULL condition	Circular: $(\text{rear} + 1) \% \text{MAX} == \text{front}$ Linear: $\text{rear} == \text{MAX} - 1$
Wrap-around	Circular: YES (modulo) Linear: NO
Applications	CPU scheduling, Traffic management, Buffer management

b) Types of Queues

1. Simple (Linear) Queue

Standard FIFO queue. Insertion at rear, deletion at front. Disadvantage: front elements' space is wasted after deletion.

Use case: Task scheduling, Print queue, Customer service line

2. Circular Queue

Last position connects back to first. Solves wasted-space problem of linear queue using modulo arithmetic.

Use case: CPU round-robin scheduling, Data buffering in networking, Keyboard buffer

3. Double Ended Queue (Deque)

Insertion and deletion can be done at BOTH ends (front and rear). Two types: Input-restricted deque, Output-restricted deque.

Use case: Undo operations, Browser cache, Palindrome checking

4. Priority Queue

Each element has a PRIORITY. Higher priority elements are dequeued first. If equal priority, FIFO order is followed.

Use case: OS process scheduling, Dijkstra's shortest path, Hospital emergency triage

Q4. a) Infix to Postfix Conversion | b) Postfix Algorithm + Evaluation [4M + 4M]**a) Infix to Postfix: $A+(B*C-(D/E^F)*G)*H$** Operator Precedence (highest to lowest): $\wedge$ (Exponent) $>$ $*$ / (Multiply/Divide) $>$ $+$ - (Add/Subtract)

Rule: Higher precedence operators are placed in output before lower precedence ones.

Infix to Postfix: $A+(B*C-(D/E^F)*G)*H$

#	Sym	Action	Stack	Output
1	A	operand	A	A
2	+	push	+	A
3	(	push	+(	A
4	B	operand	+(	AB
5	*	push	+(*	AB
6	C	operand	+(*	ABC
7	-	pop *, push -	+(.	ABC*
8	(	push	+(.(	ABC*
9	D	operand	+(.(	ABC*D
10	/	push	+(./	ABC*D
11	E	operand	+(./	ABC*DE
12	^	push	+(./ ^	ABC*DE
13	F	operand	+(./ ^	ABC*DEF
14	)	pop till (	+(.	ABC*DEF^/
15	*	push	+(.*	ABC*DEF^/
16	G	operand	+(.*	ABC*DEF^/G
17	)	pop till (	+	ABC*DEF^/G*.
18	*	push	+*	ABC*DEF^/G*.

Final Postfix: $ABC*DEF^/G*.-H*+$

Fig 7.1: Step-by-step infix to postfix conversion table

Final Postfix Expression: $ABC*DEF^/G*.-H*+$ **b) Postfix Evaluation Algorithm + Trace of $ABC*+D+$ ($A=2, B=3, C=4, D=5$)****Algorithm:**

```

EVALUATE_POSTFIX(expression):
    Initialize empty stack S
    for each symbol in expression (left to right):
        if symbol is OPERAND:
            push(S, symbol)
        if symbol is OPERATOR:
            op2 = pop(S)
            op1 = pop(S)
            result = op1 OPERATOR op2

```

```

push(S, result)
return pop(S) // Final result

```

Trace for ABC^*+D+ where $A=2$, $B=3$, $C=4$, $D=5$:

Symbol Read	Stack State
A=2 (operand)	Push 2 → Stack: [2]
B=3 (operand)	Push 3 → Stack: [2, 3]
C=4 (operand)	Push 4 → Stack: [2, 3, 4]
* (operator)	Pop 4 and 3; $3*4=12$; Push 12 → Stack: [2, 12]
+ (operator)	Pop 12 and 2; $2+12=14$; Push 14 → Stack: [14]
D=5 (operand)	Push 5 → Stack: [14, 5]
+ (operator)	Pop 5 and 14; $14+5=19$; Push 19 → Stack: [19]
END	Pop 19 → FINAL RESULT = 19

UNIT 5: TREES AND GRAPHS

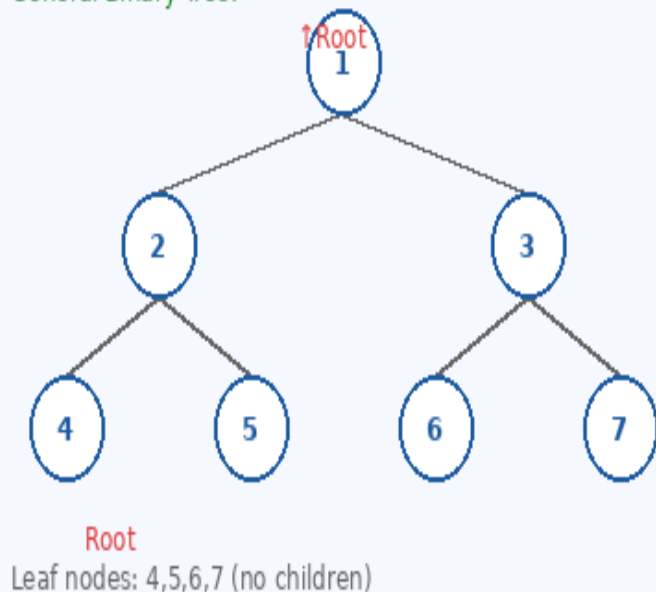
Q1. a) Binary Tree Definition & Properties | b) BST Insertion [4M + 4M]

a) Binary Tree — Definition and Properties

A Binary Tree is a hierarchical non-linear data structure in which each node has AT MOST TWO children, referred to as the LEFT child and the RIGHT child.

Binary Tree: Each node has at most 2 children

General Binary Tree:



Binary Search Tree (BST):

Left < Root < Right

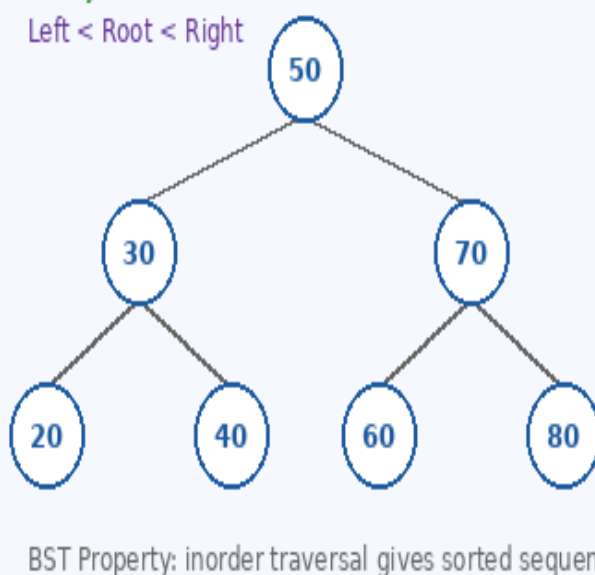


Fig 8.1: General binary tree (left) and Binary Search Tree (right)

Key Terminologies:

Term	Definition
Root	The topmost node with no parent
Leaf Node	Node with no children (terminal node)
Internal Node	Node with at least one child
Height	Number of edges on longest path from root to leaf
Depth	Number of edges from root to the node

Degree	Number of children a node has
--------	-------------------------------

Properties of Binary Tree:

1. Maximum nodes at level $i = 2^i$ (root is level 0)
2. Maximum nodes in tree of height $h = 2^{(h+1)} - 1$
3. Minimum height of tree with n nodes = $\lceil \log_2(n) \rceil$
4. Number of leaf nodes = number of nodes with 2 children + 1 (i.e., $L = T + 1$)
5. A full binary tree has all nodes with either 0 or 2 children
6. A complete binary tree has all levels filled except possibly the last, filled left to right
7. A perfect binary tree has all leaf nodes at same level

Types of Binary Trees:

- Full Binary Tree: Every node has 0 or 2 children
- Complete Binary Tree: All levels filled except last, last level filled left to right
- Perfect Binary Tree: All leaves at same level, all internal nodes have 2 children
- Degenerate Tree: Every internal node has only one child (like a linked list)

b) Insertion Operation on Binary Search Tree (BST)

A BST is a binary tree satisfying: For every node, LEFT subtree values < Node value < RIGHT subtree values. This property enables $O(\log n)$ average-case search.

Algorithm — BST Insertion:

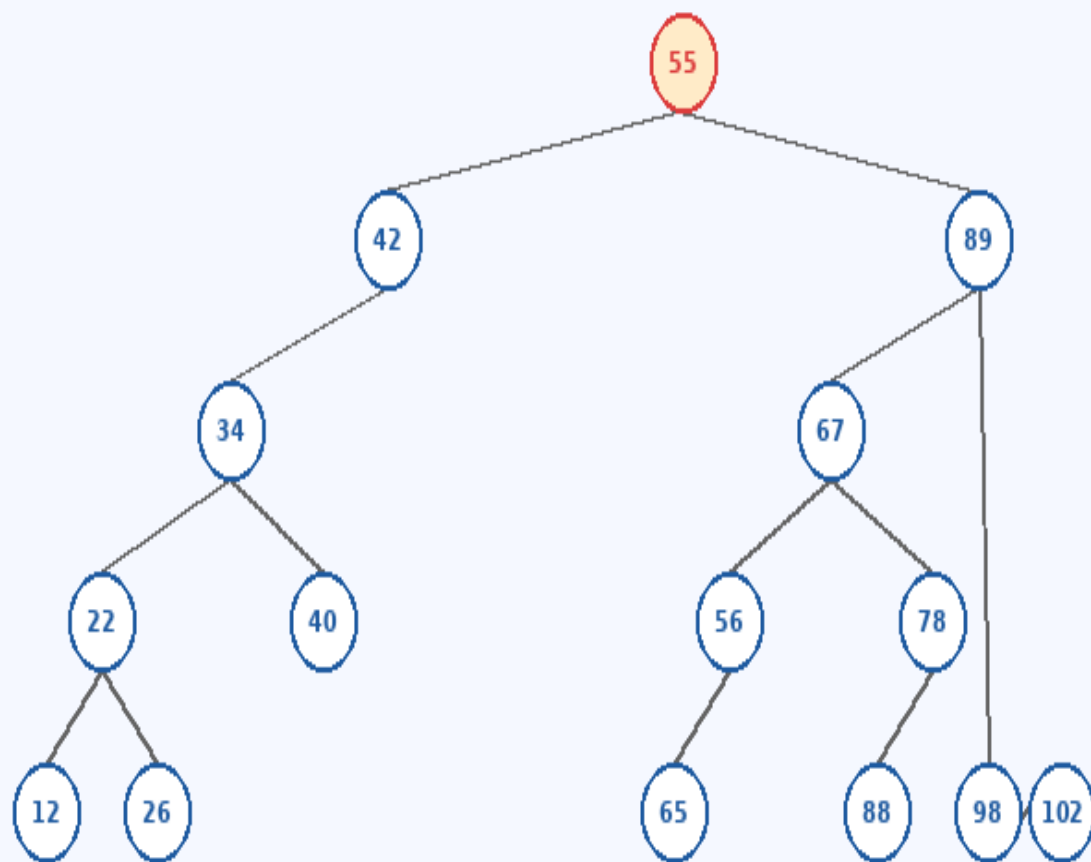
```
INSERT_BST(root, key):
    if(root == NULL):
        return createNode(key)      // New node becomes leaf
    if(key < root->data):
        root->left = INSERT_BST(root->left, key)
    else if(key > root->data):
        root->right = INSERT_BST(root->right, key)
    // If key == root->data: duplicate, do nothing
    return root
```

Example: Insert keys 50, 30, 70, 20, 40 into empty BST

Key Inserted	Action Taken
50	Tree empty → 50 becomes ROOT
30	$30 < 50 \rightarrow$ Insert in LEFT subtree of 50
70	$70 > 50 \rightarrow$ Insert in RIGHT subtree of 50
20	$20 < 50, 20 < 30 \rightarrow$ Left of 30
40	$40 < 50, 40 > 30 \rightarrow$ Right of 30

Q2. a) Construct BST for Given Keys | b) Build Tree from Inorder + Postorder [4M + 4M]

a) BST Construction: 55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

BST Construction: 55,42,89,34,67,78,40,22,12,56,88,65,26,98,102

Root=55. Keys<55 go LEFT, keys>55 go RIGHT (recursively)

Inorder traversal: 12 22 26 34 40 42 55 56 65 67 78 88 98 102 (sorted)

Fig 9.1: Final BST — root=55, all keys inserted following BST property

Insertion Trace:

Key	Insertion Path
55	ROOT
42	42<55 → LEFT of 55
89	89>55 → RIGHT of 55
34	34<55, 34<42 → LEFT of 42
67	67>55, 67<89 → LEFT of 89
78	78>55, 78<89, 78>67 → RIGHT of 67
40	40<55, 40<42, 40>34 → RIGHT of 34
22	22<55, 22<42, 22<34 → LEFT of 34
12	12<55, 12<42, 12<34, 12<22 → LEFT of 22
56	56>55, 56<89, 56<67 → LEFT of 67
88	88>55, 88<89, 88>67, 88>78 → RIGHT of 78

65	$65 > 55, 65 < 89, 65 < 67, 65 > 56 \rightarrow$ RIGHT of 56
26	$26 < 55, 26 < 42, 26 < 34, 26 > 22 \rightarrow$ RIGHT of 22
98	$98 > 55, 98 > 89 \rightarrow$ RIGHT of 89
102	$102 > 55, 102 > 89, 102 > 98 \rightarrow$ RIGHT of 98

b) Reconstruct Binary Tree from Inorder and Postorder Traversals

Inorder: {4, 8, 2, 5, 1, 6, 3, 7}

Postorder: {8, 4, 5, 2, 6, 7, 3, 1}

Key Rule

Postorder: Last element = ROOT of current subtree

Inorder: Find ROOT $\rightarrow$ elements LEFT of ROOT = left subtree | elements RIGHT = right subtree

Recurse on each subtree using the same rule

Reconstruct Tree from Inorder={4,8,2,5,1,6,3,7} Postorder={8,4,5,2,6,7,3,1}

Rule: Last element of Postorder = ROOT; find it in Inorder to split left/right subtrees

Step 1: Postorder last = 1 $\rightarrow$ ROOT=1

Inorder: {4,8,2,5} | 1 | {6,3,7} $\rightarrow$ Left subtree={4,8,2,5}, Right={6,3,7}

Step 2: Left subtree Postorder={8,4,5,2} $\rightarrow$ last=2 $\rightarrow$ ROOT of left=2

Inorder: {4,8} | 2 | {5} $\rightarrow$ 2's left={4,8}, right={5}

Step 3: {4,8} Postorder={8,4} $\rightarrow$ last=4 $\rightarrow$ ROOT=4

Inorder: {} | 4 | {8} $\rightarrow$ 4's right=8

Step 4: Right subtree Postorder={6,7,3} $\rightarrow$ last=3 $\rightarrow$ ROOT of right=3

Inorder: {6} | 3 | {7} $\rightarrow$ 3's left=6, right=7

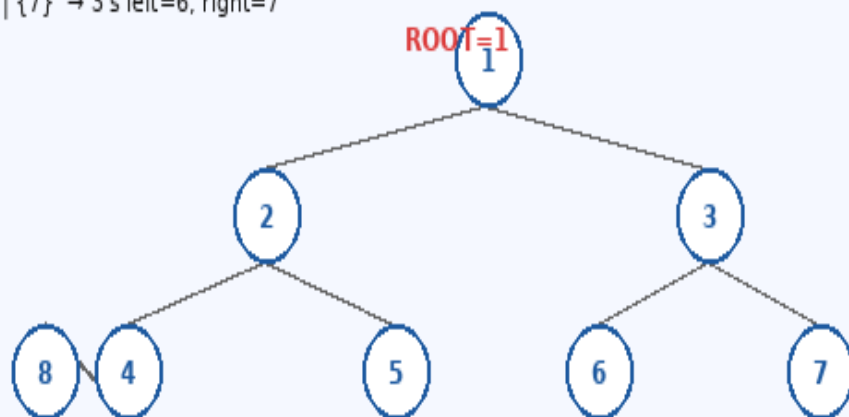


Fig 9.2: Reconstruction steps and final binary tree

Step	Details
Step 1	Postorder last = 1 $\rightarrow$ ROOT = 1. Inorder: {4,8,2,5} LEFT {6,3,7} RIGHT

Step 2 (Left)	Postorder of left = {8,4,5,2} → last=2 → ROOT=2. Inorder: {4,8} LEFT {5} RIGHT
Step 3 (Left-Left)	Postorder {8,4} → last=4 → ROOT=4. Inorder: {} LEFT {8} RIGHT. So 8 is right child of 4
Step 3 (Left-Right)	Only {5} → ROOT=5, leaf node
Step 4 (Right)	Postorder of right = {6,7,3} → last=3 → ROOT=3. Inorder: {6} LEFT {7} RIGHT
Final Tree	Root=1; L=2(L=4(R=8), R=5); R=3(L=6, R=7)

Q3. a) Graph Definition & Representations | b) BFS Traversal [4M + 4M]

a) Graph Definition and Representation Methods

A Graph $G = (V, E)$ is a non-linear data structure consisting of:

- V: A set of VERTICES (also called nodes)
- E: A set of EDGES (connections/relationships between vertices)

Types of Graphs:

- Undirected Graph: Edges have no direction. Edge $(u,v) = (v,u)$
- Directed Graph (Digraph): Edges have direction. $(u,v) \neq (v,u)$
- Weighted Graph: Each edge has a weight/cost
- Connected Graph: There is a path between every pair of vertices

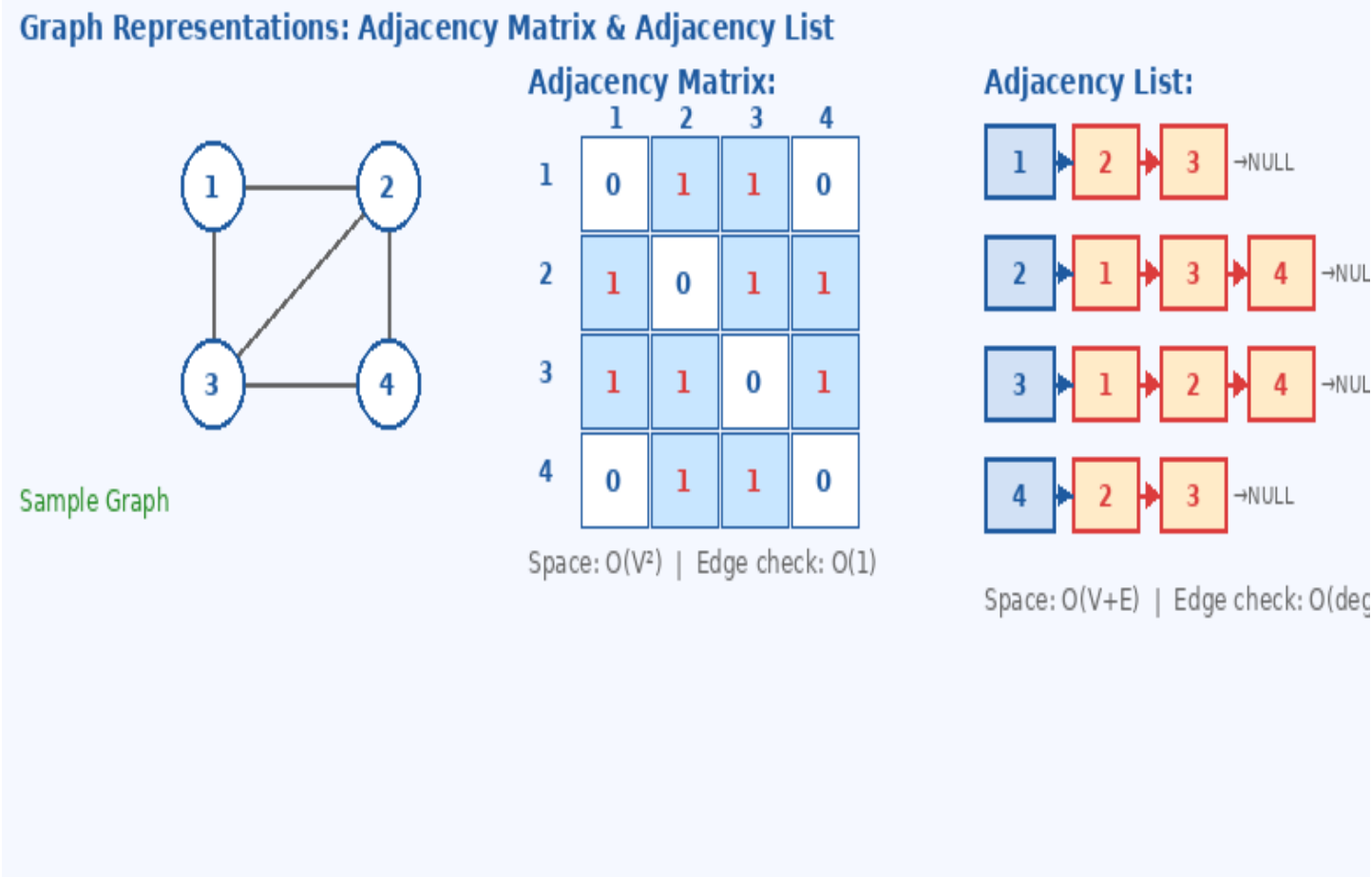


Fig 10.1: Graph with Adjacency Matrix (left) and Adjacency List (right)

1. Adjacency Matrix Representation:

A 2D array A of size $V \times V$ where $A[i][j] = 1$ if edge exists between vertex i and j, else 0.

- Space Complexity: $O(V^2)$
- Edge lookup: $O(1)$ — constant time
- Best for: Dense graphs (many edges)
- Disadvantage: Wastes space for sparse graphs

2. Adjacency List Representation:

An array of linked lists. For each vertex, a list of all adjacent vertices is maintained.

- Space Complexity: $O(V + E)$
- Edge lookup: $O(\text{degree of vertex})$
- Best for: Sparse graphs (few edges)
- Advantage: Space-efficient, easy to find all neighbors

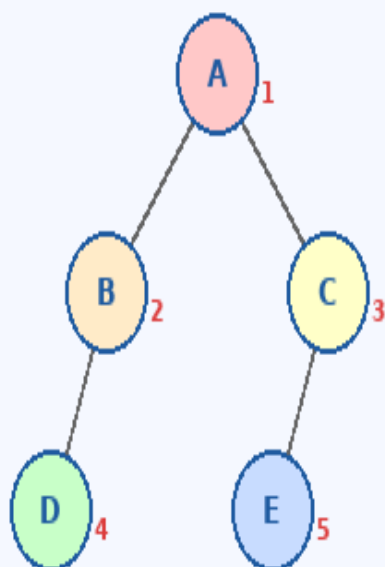
Feature	Adjacency Matrix vs Adjacency List
Space	Matrix: $O(V^2)$ List: $O(V+E)$

Edge Check	Matrix: $O(1)$ List: $O(\text{degree})$
Add Edge	Matrix: $O(1)$ List: $O(1)$
Best For	Matrix: Dense graphs List: Sparse graphs
Memory	Matrix: High List: Low

b) BFS Traversal using Queue

Data Structure Used: QUEUE (FIFO) — BFS visits all neighbors of a node before going deeper.

BFS (Breadth First Search) - Uses QUEUE - Level-by-level traversal



Visit order: A→B→C→D→E

BFS Queue Trace (Start from A):

Init

Enqueue A

Queue:[A] Visited:{}

Visit A

Dequeue A, enqueue B,C

Queue:[B,C] Visited:{A}

Visit B

Dequeue B, enqueue D

Queue:[C,D] Visited:{A,B}

Visit C

Dequeue C, enqueue E

Queue:[D,E] Visited:{A,B,C}

Visit D

Dequeue D (no unvisited)

Queue:[E] Visited:{A,B,C,D}

Visit E

Dequeue E (no unvisited)

Queue:[] Visited:{A,B,C,D,E}

BFS uses a Queue. Data structure: QUEUE (FIFO)

Fig 10.2: BFS traversal — level-by-level, uses Queue

BFS Algorithm:

```

BFS(graph, startVertex):
  Create empty Queue Q and Visited[] array
  Mark startVertex as visited
  Enqueue startVertex into Q
  while Q is not empty:
    vertex = Dequeue(Q)
    print vertex           // Process vertex
  
```

```

for each neighbor w of vertex:
    if w is NOT visited:
        mark w as visited
        Enqueue(w) into Q

```

Time Complexity: $O(V + E)$ | Space Complexity: $O(V)$ (for queue and visited array)

Applications: Shortest path in unweighted graphs, Web crawlers, Social network friend suggestions, GPS navigation

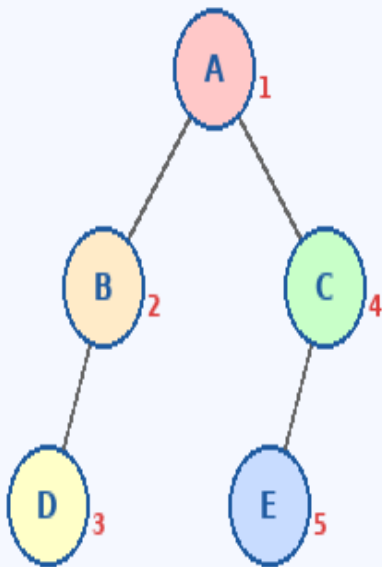
Q4. DFS and BFS Algorithms with Trace [8M]

Both DFS and BFS are fundamental graph traversal algorithms used to visit all vertices of a graph.

A) Depth First Search (DFS)

DFS explores as FAR as possible along each branch before backtracking. It uses a STACK (or recursion).

DFS (Depth First Search) - Uses STACK (or Recursion) - Go deep first



DFS order: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

DFS Stack Trace (Start from A):

Init

Push A
Stack:[A] Visited:{}

Visit A

Pop A, push C,B
Stack:[C,B] Visited:{A}

Visit B

Pop B, push D
Stack:[C,D] Visited:{A,B}

Visit D

Pop D (no unvisited)
Stack:[C] Visited:{A,B,D}

Visit C

Pop C, push E
Stack:[E] Visited:{A,B,D,C}

Visit E

Pop E (no unvisited)
Stack:[] Visited:{A,B,D,C,E}

DFS uses a Stack. Data structure: STACK (LIFO)

Fig 11.1: DFS traversal — goes deep first, uses Stack (or recursion)

DFS Algorithm:

```
DFS(graph, startVertex):
```

```

Create Stack S and Visited[] array
Push startVertex onto S
while S is not empty:
    vertex = Pop(S)
    if vertex NOT visited:
        mark vertex as visited
        print vertex
        for each neighbor w of vertex (in reverse order):
            if w NOT visited:
                Push(S, w)

// Recursive DFS:
DFS_RECURSIVE(vertex, Visited[]):
    Visited[vertex] = true
    print vertex
    for each neighbor w of vertex:
        if NOT Visited[w]:
            DFS_RECURSIVE(w, Visited[])

```

B) Breadth First Search (BFS)

BFS explores all neighbors at the current level before moving to the next level. It uses a QUEUE.

```

BFS(graph, startVertex):
    Create Queue Q and Visited[] array
    Visited[startVertex] = true
    Enqueue(Q, startVertex)
    while Q is not empty:
        vertex = Dequeue(Q)
        print vertex
        for each neighbor w of vertex:
            if NOT Visited[w]:
                Visited[w] = true
                Enqueue(Q, w)

```

Comparison of DFS vs BFS:

Feature	DFS vs BFS
Data Structure	DFS: Stack BFS: Queue
Traversal Order	DFS: Depth-first (one path fully) BFS: Breadth-first (level by level)
Time Complexity	Both: $O(V + E)$
Space Complexity	DFS: $O(V)$ recursive stack BFS: $O(V)$ queue
Shortest Path	DFS: NOT guaranteed BFS: Finds shortest path (unweighted)
Applications	DFS: Topological sort, cycle detection BFS: Shortest path, web crawling
Completeness	DFS: May get stuck in infinite loops BFS: Always finds solution if exists

Trace on Sample Graph (A–B–C–D–E):

Graph edges: A-B, A-C, B-D, C-E. Starting vertex: A

Algorithm	Traversal Order
DFS	A → B → D → C → E (goes deep via B first)
BFS	A → B → C → D → E (visits all A's neighbors first)

Summary: When to use DFS vs BFS?

Use DFS when: Checking connectivity, topological sorting, detecting cycles, solving mazes

Use BFS when: Finding shortest path, social network analysis, web crawling (level-by-level)

Memory tip: DFS = Depth = Stack | BFS = Breadth = Queue